



SAPIENZA
UNIVERSITÀ DI ROMA

Advanced Software Engineering

NNModelling

A Visual DSL for Neural Network Design

Davide Pietragalla & Luca Sforza

24 giugno 2026

- 1 Introduction
- 2 Requirements
- 3 Design
- 4 Implementation
- 5 Validation
- 6 Conclusions

- **NNModelling** is a DSL for designing neural network architectures using a visual diagrammatic language
- Models are compiled into Hydra configuration files that dynamically instantiate a working **PyTorch Lightning** model
- We adopted a **model-driven** approach: UML metamodel → front-end editor → back-end pipeline

Core pipeline:

Visual Graph → NNTree → JSON → Hydra YAML → LightningModule

Functional Requirements

- Visual Diagram Editor
- Node Types (module, input, join, subflow)
- Implicit Forks
- Explicit Joins
- Extensible Module Registry
- Subflow Containers
- Behavioral Stereotypes
- Parameter Configuration
- Config Generation
- Dynamic Model Instantiation
- Training Pipeline

Non-Functional Requirements

- **Separation of Concerns:** front-end handles DSL logic, back-end handles instantiation
- **Test Coverage:** comprehensive tests across both subsystems
- **Expressiveness:** capable of modeling transformers, residual connections, etc.
- **Execution Correctness:** BFS topological traversal with in-degree tracking

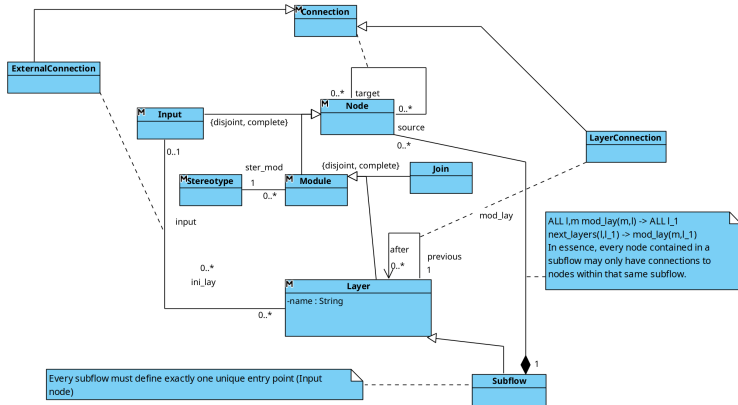


Figura: Node hierarchy: Module, Input, Join, Subflow with recursive composition.

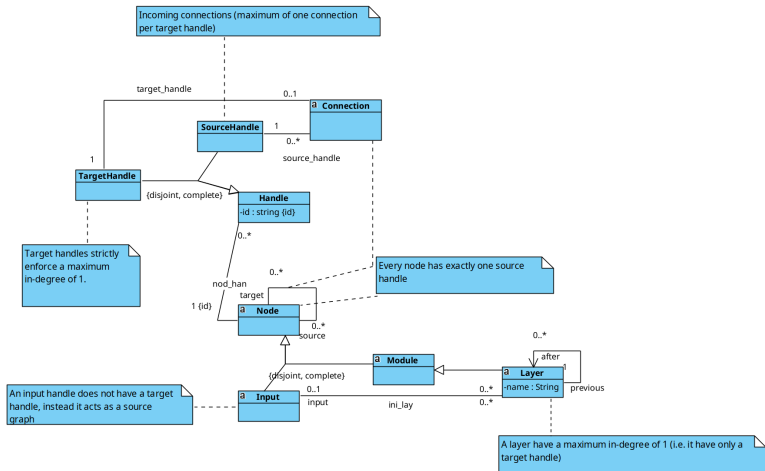


Figura: Handle-based connections: implicit forks and explicit joins.

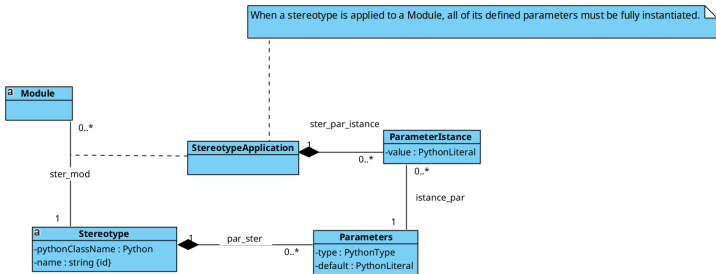


Figura: Stereotype system: separation between definition and instantiation.

Formally, a graph $G = (V, E)$ is compiled into a tree $T = (N, C)$

Transformations:

- Maximal linear chains $\rightarrow$ Sequential
- Merge points ($\text{indegree} > 1$) $\rightarrow$ Join
- Subflows $\rightarrow$ recursive compilation
- Loss nodes $\rightarrow$ stored separately

Result: an **execution-oriented** representation independent of any DL framework

Front-End

- Svelte 5, Svelte Flow, TypeScript
- Visual DSL editor (browser-only)
- Compiler: graph $\rightarrow$ NNTree $\rightarrow$ JSON

Back-End

- Python, PyTorch Lightning, Hydra
- Config generator (7 YAML files)
- Dynamic model instantiation

Communication via JSON file exchange

- **27 modules:** Linear, Conv2d, ReLU, Tanh, Embedding, LayerNorm, Dropout, etc.
- **6 joins:** Addition, Concat, Einsum, MatMul, ScaledDotProduct, MaskedScaledDotProduct
- **2 subflow templates:** Repeat (sequential), HorizontalRepeat (parallel via vmap)
- **Custom ops:** PositionalEncoding, SequencePool

- LightningModule builds architecture from Hydra config at runtime
- Forward pass uses BFS traversal with in-degree tracking
- Join nodes collect all inputs ordered by targetHandle
- Custom operations: Subflow, Repeat, HorizontalRepeat

Result: 179 tests across front-end and back-end

Example: Transformer Classifier

Three-level Flow for Building the Encoder:

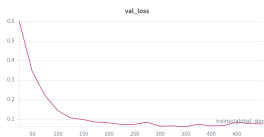
- Top-level: Input $\rightarrow$ Embedding $\rightarrow$ PosEnc $\rightarrow$ Transformer $\rightarrow$ Pool $\rightarrow$ Linear $\rightarrow$ Loss
- Repeat encoder with residual blocks and FFN
- HorizontalRepeat multi-head attention with Q/K/V projections

The architecture is taken from "Attention Is All You Need" paper.

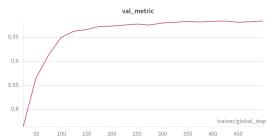
Result: 98.4% accuracy on EnronSpam test set.



(a) Training loss



(b) Validation loss



(c) Validation accuracy

- Full end-to-end pipeline from visual design to trained model
- Extensible stereotype system – no code changes for new layers
- Complex architectures: transformers, residual nets, subflows
- 179 tests ensuring reliability
- Model-driven approach with formal UML metamodel
- Clean separation of concerns

- **Network Communication:** replace file exchange with HTTP/WebSocket API
- **Configurable Join:** parameterize HorizontalRepeat merge strategy
- **Parallel Execution:** CUDA stream dispatcher for fork-join topologies
- **Subflow Persistence:** save/load subflows independently
- **Interactive Deployment:** HuggingFace Spaces
- **Pre-trained Models:** transfer learning support

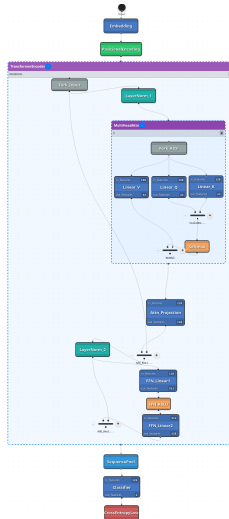


Figura: Full transformer classifier graph in the NNModelling editor

-  Svelte 5 Documentation. <https://svelte.dev/docs>
-  Svelte Flow Documentation. <https://svelteflow.dev/>
-  Paszke, A. et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” NeurIPS 2019.
-  Falcon, W. et al. “PyTorch Lightning.” <https://lightning.ai/>
-  Vaswani, A. et al. “Attention Is All You Need.” NeurIPS 2017.
-  Yadan, O. “Hydra - A Framework for Elegantly Configuring Complex Applications.” <https://hydra.cc/>